数学与 CS 问答笔记

answer-note

0101 当前作业的三层问题：契约（未命中算不算错误）在规格、测试、实现三处说法不一致；实现层的 Set 顺序与 Get 的 copy；规格层 AF / RI / Safety 与方法 doc 的四处缺口；以及断言粒度与工程卫生。每条都附本机可查的真实项目样本。

November 07, 2026

Contents

1. 0101 当前作业的问题清单：规格 / 测试 / 实现三层的缺口｜问题 0101 5
   1.1. 费曼直觉 5
   1.2. 硬核机制 5
      1.2.1. 一、契约不一致：同一个行为有三种说法 5
      1.2.2. 二、实现层：三处直接让测试变红的错误 6
      1.2.3. 三、规格层：AF / RI / Safety 各缺什么 6
      1.2.4. 四、测试层：三个 TODO 与断言粒度 8
      1.2.5. 五、工程卫生：三个与代码无关但会咬人的点 9
   1.3. 陷阱与易错 9
   1.4. 关键点 10
   1.5. 变式检验 10
   1.6. 同类归类 10
   1.7. Go 语言机制补充 11
   1.8. 本次操作与当前状态 11
2. 用 Delve 调试 kv_test.go 的完整工作流（Debug Workflow）｜问题 0101 13
   2.1. 费曼直觉 13
   2.2. 硬核机制 13
      2.2.1. 第 0 步：红点是入口，先分类再动手 13
      2.2.2. 第 1 步：在失败的那条断言上断点 14
      2.2.3. 第 2 步：钻进实现，分清 args、locals 与返回值 14
      2.2.4. 第 3 步：条件断点，把重复调用过滤掉 15
      2.2.5. 第 4 步：验证「顺序类」bug，用 next 走一遍副作用 16
      2.2.6. 第 5 步：调用栈、display 与 restart 17
      2.2.7. 第 6 步：把会话变成可重放的脚本 18
      2.2.8. 工作流全图 18
   2.3. 陷阱与易错 18
   2.4. 关键点 19
   2.5. 变式检验 19
   2.6. 同类归类 19
   2.7. Go 语言机制补充 20
   2.8. 本次操作与当前状态 20
3. 在 Neovim 里调试 kv_test.go（nvim-dap + nvim-dap-go）｜问题 0101 21
   3.1. 费曼直觉 21
   3.2. 硬核机制 21
      3.2.1. 一、本机现状（已核实） 21
      3.2.2. 二、按一下键，它到底做了什么 21
      3.2.3. 三、一次完整会话（按你现在的 kv_test.go） 23
      3.2.4. 四、条件断点：只在 k1 上停 23
      3.2.5. 五、调试入口不止一个 24
      3.2.6. 六、真实项目里这套配置长什么样 24
   3.3. 陷阱与易错 24

3.4. 关键点 25
3.5. 变式检验 25
3.6. 同类归类 26
3.7. Go 语言机制补充 26
3.8. 本次操作与当前状态 26
4. dlv test 命令的参数归谁解析：分界符与测试标志｜问题 0101 28
4.1. 费曼直觉 28
4.2. 硬核机制 28
4.2.1. 一、逐段拆解 28
4.2.2. 二、`--` 之前：dlv 自己的部分 29
4.2.3. 三、`--` 之后：测试二进制的部分 29
4.2.4. 四、为什么必须有 `--`：反面实测 30
4.2.5. 五、为什么是 `-test.v` 而不是 `-v` 31
4.2.6. 六、`-other-argument` 能换成什么 31
4.3. 陷阱与易错 31
4.4. 关键点 32
4.5. 变式检验 32
4.6. 同类归类 33
4.7. Go 语言机制补充 33
4.8. 本次操作与当前状态 33
5. 为什么 delve 的 `list` 显示 `no source available`：PC 落在了没有行表的地址上｜问题 0101 34
5.1. 费曼直觉 34
5.2. 硬核机制 34
5.2.1. 一、为什么启动瞬间会停在「没有源码」的地方 34
5.2.2. 二、对照实测：同一个 `list`，换个时机就正常 35
5.2.3. 三、Go 的汇编函数仍然有源码 35
5.2.4. 四、按这个思路排查的顺序 36
5.2.5. 五、如果 `continue` 之后还是 `no source available` 36
5.3. 陷阱与易错 37
5.4. 关键点 37
5.5. 变式检验 37
5.6. 同类归类 38
5.7. Go 语言机制补充 38
5.8. 本次操作与当前状态 38
6. 怎么按函数名设断点：`break` 的 locspec 与把函数名参数化｜问题 0101 39
6.1. 费曼直觉 39
6.2. 硬核机制 39
6.2.1. 一、函数名的五种写法（全部本机实测） 39
6.2.2. 二、`:行偏移`：进函数后第几行 40
6.2.3. 三、命名断点：给断点起名，之后用名字操作 40
6.2.4. 四、一次设一批：正则 locspec 41

6.2.5. 五、真正把函数名做成「参数」：脚本与 shell 函数 41
6.2.6. 六、Neovim 里没有「函数名断点」这回事 42
6.3. 陷阱与易错 42
6.4. 关键点 43
6.5. 变式检验 43
6.6. 同类归类 43
6.7. Go 语言机制补充 44
6.8. 本次操作与当前状态 44
Index of Listings 45

1. 0101 当前作业的问题清单：规格 / 测试 / 实现三层的缺口 | 问题 0101

1.1. 费曼直觉

这份作业现在的问题，重点不在「哪里写错了」，而在「三份文件讲同一个行为时说法不一致」。

把作业想成装修：`kv.go` 顶部的 AF / RI / Safety 是**合同**，`kv_test.go` 顶部的 testing strategy 与断言是**验收单**，`kv.go` 里的三个方法体是**施工**。合同写「开关朝上」，验收单写「开关朝下」，工人装成了「没装开关」——三方各自看都合理，合起来必然吵。

所以评审的顺序不是「从第一行往下挑错」，而是三问：

- 合同与验收单是否说的是同一件事？(契约)
- 施工是否兑现了合同？(实现)
- 验收单本身是否覆盖了合同写死的每一条？(测试)

> 📌 **核心要点** 三层里**契约**最贵：契约不定，改实现是白改，补测试也是白补。当前最紧的一件事是回答「未命中算不算错误」，而不是先把 `copy` 补上。

1.2. 硬核机制

1.2.1. 一、契约不一致：同一个行为有三种说法

场景	规格注释怎么说	测试怎么要求	实现怎么做
Get 未命中	第 41 行注释写着 `(nil, false, nil)`	第 42、50 行 `!ok and err == nil`	`ErrNotFound`
Del 未命中	没有方法规格	第 45 行 `!updated and err == nil`	`ErrNotFound`
Set 同键同值	没有方法规格	没有用例	`ErrUpdatedFailed`

两个包级变量 `ErrNotFound`、`ErrUpdatedFailed` 现在的真实身份是：**测试不接受、规格没写、实现却在用的错误类型**。

标准库在这个岔口上选了另一条路。本机 `go doc -src sync.Map.Load` 的第一句就是：

```
// Load returns the value stored in the map for a key, or nil if no
// value is present.
// The ok result indicates whether value was found in the map.
func (m *Map) Load(key any) (value any, ok bool)
```

Listing 1: 未命中用 ok 表达、不产生 error（sync/map.go）

bbolt 走的是第三种：`Bucket.Get` 查不到直接返回 `nil`，文档原文 “Returns nil if the key does not exist in the bucket”（`https://pkg.go.dev/go.etcd.io/bbolt#Bucket.Get`，未在本机验证）。

三条路线都合法，但**同一个项目里只能选一条**。你的测试已经选了 `ok bool` 路线，那么正确做法是把 `ErrNotFound` / `ErrUpdatedFailed` 删掉，或明确它们只在别的场合使用。

> ⚠️ **陷阱预警** 最容易走偏的一步：为了让实现「用上」 `ErrNotFound` ，去改测试第 42 行。测试是你先写的验收标准，改它的代价是「把验收单改成能过」，而不是「把活干对」。要么删 error 变量，要么在规格里改契约并同步改测试——两件事一起做，不能只改一边。

1.2.2. 二、实现层：三处直接让测试变红的错误

当前 `Set` 的代码就是问题最集中的地方：

```
    kv.mem[string(key)] = make([]byte, len(val))        // ① 先写
    updated = bytes.Equal(val, kv.mem[string(key)])    // ② 再比：比的是刚写进去的零切片
    if !updated {
      err = ErrUpdatedFailed                            // ③ 没变被当成错误
    }
```

Listing 2: kv.go 第 51 至 55 行的现状

三处独立的问题：

- **顺序反了**：比较必须在写入之前，否则永远是「自己跟自己比」;
- **`make` 只分配不拷贝**： `make([]byte, n)` 给出长度为 `n` 的全零切片，mem 里存的就是零字节。这和 `Get` 返回 `[0,0]` 是同一个根因，昨天用 Delve 已经实证；
- **「值没变」被当错误**： `updated=false` 是正常返回值，讲义要求「同键同值返回 false」，不是失败。

顺序骨架（这不是完整实现；注意 ① 得到的 `ok` 与 `old` 正是 ③ 的输入）：

```
    // ① 读：old, ok := kv.mem[string(key)]
    // ② 算：updated 由 ok 与 old 得到（你文件里的 TODO 注释已给出式子）
    // ③ 写：kv.mem[string(key)] 存一份副本，与 Safety 的承诺一致
    // ④ 返回：updated, nil（值没变也返回 nil）
```

Listing 3: Set 的四步顺序：读、算、写、返回

`Get` 的病灶只有一行： `make` 之后没有 `copy` 。 `Del` 的一处病灶是未命中时返回了 `ErrNotFound` ，与第 45 行冲突。

1.2.3. 三、规格层：AF / RI / Safety 各缺什么

条目	现在写的	缺的	不补会怎样
AF	`{ k -> mem[k] | all k in mem }`	抽象域是 `[]byte -> []byte` 的**偏函数**；内部键是 `string` ， `string(key)` 的对应关系与「字节内容相同即同一个键」没写	客户不知道两个内容相同的 `[]byte` 是同一个键

RI	`len( each of mem.keys) >= 1` 两条	① `mem` 非 nil、只在 `Open` 之后可用；② 这两句不是可求值表达式；③ `checkRep` 写在哪里	直接 `KV{}` 就 `Set` 会 panic
Safety	一句话：Get/Set 做防御性拷贝	入向（Set 存副本）与出向（Get 返副本）分开写；Del 无需拷贝也要说明；且要与代码对齐	注释承诺的拷贝在代码里不存在
方法规格	完全没有	每个方法的 doc comment：未命中返回什么、`updated` 的双重含义、`Close` 会不会失败	契约只活在测试里，换个测试就失效

⚠ **陷阱预警** `len( each of mem.keys) >= 1` 这句话会被读成「map 至少有一个键」。Go 里 `mem.keys` 根本不存在，`mem` 的键是 `string`。抽象表示里没有「键集」这个字段，只有 `map[string][]byte` 一个字段；要表达「每个键的字节长度至少 1」，写成 `for all k in dom(mem): len(k) >= 1` 才没有歧义。

RI 缺 `mem` 非 nil 这一条不是纸面问题。`KV{}` 的零值上直接 `Set` 会 panic，本机探针实测（`$TMPDIR/probe/main.go`）：

```
    kv := KV{}                  // 没有调用 Open
    defer func() { recover(); println("recovered") }()
    kv.Set([]byte("k"), []byte("v"))   // panic: assignment to entry in nil
map
```

Listing 4: 零值未初始化时写 map 的实测结果

运行时出处：`/usr/lib/go/src/runtime/map.go` 第 21 行定义了 `maps_errNilAssign`，写 nil map 时抛出该 panic。

Safety 的两条路线，在真实项目里各有一个代表：

路线	真实项目样本	文档里必须写的那句
声明别名、不拷贝	`bytes.Buffer.Bytes`：`return b.buf[b.off:]`	「The slice aliases the buffer content」，并说明有效期到下一次修改
出向拷贝	`strings.Clone` / groupcache `ByteView.ByteSlice`	「returns a copy」，客户可随意修改
入向拷贝	`net/http.Header.Set`：每次 `[]string{value}`	「caller may reuse the argument」，实现不持有调用方的切片

本机 `go doc -src bytes.Buffer.Bytes` 的原文（节选）：

```
    // Bytes returns a slice of length b.Len() holding the unread portion of
the buffer.
    // The slice is valid for use only until the next buffer modification.
    // The slice aliases the buffer content at least until the next buffer
modification,
    // so immediate changes to the slice will affect the result of future
reads.
    func (b *Buffer) Bytes() []byte { return b.buf[b.off:] }
```

Listing 5: 不拷贝也可以，但必须把「别名」与「有效期」写进文档

你的 Safety 注释选的是「出向 + 入向都拷贝」，这是安全但更贵的一条。选它没问题，问题在于代码还没兑现。

1.2.4. 四、测试层：三个 TODO 与断言粒度

三个 TODO 已经在 11-05 第三轮评审给过写法（Set 三个子域、history、断言映射与未覆盖清单），它们属于**未落盘**的工作，而不是「已经会了但没写」。

另一个没被评审点名的、独立的问题是断言粒度：

```
    assert.True(t, string(val) == "v1" && ok && err == nil)
```

Listing 6: kv_test.go 第 39 行：三个条件焊成一条断言

失败时 testify 只输出 `Should be true`。三个条件里哪个错、`val` 到底是什么内容，一概看不到——昨天正是因此才要上 Delve，才看到 `ok` 是 true、`err` 是 nil、`val` 是 `[0,0]`。

标准库的写法是每个条件一条消息、每条消息带 got 与 want（`/usr/lib/go/src/strconv/number_test.go`）：

```
    var atobtests = []atobTest{
      {"", false, ErrSyntax},
      {"asdf", false, ErrSyntax},
      {"0", false, nil},
      {"false", false, nil},
      {"true", true, nil},
    }

    if e != nil {
      t.Errorf("ParseBool(%s) = %s; want nil", test.in, e)
    }
    if b != test.out {
      t.Errorf("ParseBool(%s) = %t; want %t", test.in, b, test.out)
    }
```

Listing 7: 每个 case 三个字段，断言分开写，失败消息自带 got 与 want

对应的 Go 版本形状（testify）：

```
    require.NoError(t, err)              // 失败即停，后续断言没有意义
    assert.True(t, ok)
    assert.Equal(t, []byte("v1"), val)   // 失败时打出 diff，而不是一句 Should
be true
```

Listing 8: 把三合一断言拆成三条，失败点一眼可见

注意 `atobtests` 的第一行 `{"", false, ErrSyntax}`：空输入有专门的期望值。你的 RI 写了「键长度至少 1」，测试里却没有任何空键用例，这一条正好对应你标了 `TODO(阶段末)` 的待办。

1.2.5. 五、工程卫生：三个与代码无关但会咬人的点

问题	现状	代价
没有任何提交	`git log` 只有 `init project` 一条	`note/`、九个 `*_test.go`、`AGENTS.md` 全部 untracked；改坏了无法回滚、无法对比
`scratch/` 没被忽略	`.gitignore` 里没有 `scratch/`	`go build ./...` 会把它一起构建，`git add .` 会把实验代码提交进去
索引没同步	`INDEX.md` 只记到 11-05 的前 6 节	后 4 节（含 Delve）查不到

1.3. 陷阱与易错

⚠️ **陷阱预警 `make` 不是 `copy`。** `make([]byte, len(val))` 只保证长度，内容全是零值。拷贝有两条常见写法：`out := make([]byte, len(val)); copy(out, val)`，或 `out := append([]byte(nil), val...)`。前者容量精确，后者少一行；两者都不会与源共享底层数组。

⚠️ **陷阱预警「值没变」不是错误。** `Set` 返回 `(false, nil)` 是讲义写死的行为。把它当错误返回，等于让客户用 `err != nil` 去判断「业务上的没变化」，而 `err != nil` 在 Go 里的语义是「操作失败」。

⚠️ **陷阱预警 包级未使用的变量，编译器不会提醒。**探针里我写了一个从未被引用的 `ErrUnused`，`go run` 照样编译通过。Go 只对**函数内声明的局部变量未使用**报错；包级声明可能被同包其他文件使用，编译器无法判定。要抓这类残留，得靠 `staticcheck` 的 `U1000`（本机未安装）。

⚠️ **陷阱预警 合并断言会把调试成本推给调试器。** `assert.True(t, a && b && c)` 在失败时丢掉的信息，正是你昨天要花半小时在 Delve 里捞回来的东西。断言写法是「给未来的自己省时间」，不是风格问题。

1.4. 关键点

按「改一处要连带改几处」排序，工作量从小到大：

顺序	先做什么	判据	连带影响
1	定契约：未命中用 `ok bool`，删除两个 error 变量，或反过来改测试并写方法规格	`go doc` 能查到 `Get` 的未命中行为	决定 2 与 3 怎么写
2	改 `Set` 顺序（读、算、写、返回）与 `Get` 的 `copy`	`go test -run TestKVBasic` 全绿	无需动测试
3	补 `Set` 的三子域与 history 用例	`TODO` 从 3 降到 1	会新增红点，属正常
4	补 Safety 两用例（改返回值、改入参）	两条断言能过	可能需要改实现
5	补方法规格与 RI 的 `mem` 非 nil	`go vet` 仍绿，`KV{}` 的 panic 有规格依据	无

📌 **核心要点** 判据只有一句：**每一处「测试要求的」都必须能在规格里找到出处，每一处「规格承诺的」都必须能在测试里找到用例。**三者对齐之前，绿了也不代表对。

1.5. 变式检验

只给判断题，不给答案：

- 若把契约改成「Get 未命中返回 `ErrNotFound`」，7 条断言里哪几条必须改？`ok` 这个返回值还要不要？
- 若 `Del` 未命中返回 `ErrNotFound`，第 45 行怎么改才仍然能表达「没有这个键」？
- `Set` 返回 `updated=false` 时应不应该同时返回 `err != nil`？标准库里有没有反例（`sync.Map.LoadOrStore` 的 `loaded` 与 `error` 是什么关系）？
- 若把 Safety 改成「声明别名、不拷贝」（像 `bytes.Buffer.Bytes` 那样），7 条断言里有几条仍然能过？缺哪条文档声明？

1.6. 同类归类

问题类别	别的项目怎么做	出处

未命中怎么表达	`ok bool` / `nil` / `error` 三选一，同一项目内一致	`sync.Map.Load`；bbolt `Bucket.Get`
拷贝与否	要么拷贝，要么在文档里声明别名与有效期	`bytes.Buffer.Bytes`；`net/http.Header.Set`
未使用符号	编译器不管包级，交给 `staticcheck` 之类的 linter	`honnef.co/go/tools` 的 `U1000`
断言粒度	一个条件一条消息，带 got 与 want	GOROOT `strconv/number_test.go`；`testing` 包的 `Errorf` 惯例
初始化前置条件	类型注释里写「零值不可用，必须调用 Open」，或提供 `New` 构造函数	`sync.Mutex` 的零值可用；`os.File` 必须 `Open` 后才可用

1.7. Go 语言机制补充

- **nil map 写入**：`map` 是引用类型，零值是 `nil`。读 nil map 合法（返回零值），写 nil map 触发 `maps_errNilAssign`（`/usr/lib/go/src/runtime/map.go` 第 21 行）。所以 `Open` 里的 `kv.mem = map[string][]byte{}` 是必需的初始化，RI 应当把「已初始化」写进去，或让 `Set` / `Del` 在 nil 时自动初始化。
- **`make` 与 `append` 的区别**：`make([]byte, n)` 分配长度 `n`、容量 `n` 的零值切片；`append(s, v...)` 在容量不足时重新分配并复制。`append([]byte(nil), val...)` 恰好等价于「分配一份精确拷贝」。
- **未使用变量的规则来自编译器，不是语言规范**：Go 规范没有「包级声明必须被使用」这一条；`cmd/compile` 只对函数体内的未使用局部变量报 `declared and not used`。
- **`error` 的哨兵值与 `errors.Is`**：`ErrNotFound` 这类包级哨兵变量本身是标准做法（`io.EOF`、`os.ErrNotExist`），可用 `errors.Is` 判断。它的错不在「存在」，而在与测试契约冲突。
- **`require` 与 `assert`**：`testify/require` 失败时调用 `t.FailNow()`，后续断言不再执行；`testify/assert` 只记录失败继续跑。前者适合「后续断言依赖这一步成功」的场合（如 `Open`、`err` 检查）。

1.8. 本次操作与当前状态

- 读取 `internal/db/kv.go`（70 行）、`internal/db/kv_test.go`（54 行）、`.gitignore`、`git log` 与 `git status`；未改动 `internal/db` 任何文件。
- 实跑 `go build ./...`：通过；`go vet ./...`：通过（两者都不编译 `_test.go` 的断言逻辑，绿不代表测试对）。
- 实跑 `go test -count=1 ./internal/db/`：**红**，`TestKVBasic` 在第 36、39、42、45、51 行失败，第 32 行通过。
- 本机验证的样本：`go doc -src sync.Map.Load`、`go doc -src bytes.Buffer.Bytes`、`go doc -src net/http.Header.Set`、`/usr/lib/go/src/strconv/number_test.go`、`/usr/lib/go/src/runtime/map.go` 第 21 行。

- 探针写在会话临时目录 `$TMPDIR/probe/main.go`（nil map panic、包级未使用变量），未进仓库。
- 当前 `kv_test.go` 仍有 3 个 `TODO`；`kv.go` 仍有 8 处 `Step` 注释未兑现。
- 待本人动手：① 定契约（删 error 变量或改测试，二选一并写进方法注释）；② 改 `Set` 顺序与 `Get` 的 `copy`；③ 补 `Set` 三子域与 history；④ 补 Safety 两用例；⑤ 补 `mem` 非 nil 的 RI 条目。完成后跑 `go test -count=1 ./internal/db/` 与 `go vet ./...` 各一次。
- 建议首次提交：`git add -A` 之前先决定 `scratch/` 是否进 `.gitignore`。

2. 用 Delve 调试 kv_test.go 的完整工作流（Debug Workflow）｜问题 0101

2.1. 费曼直觉

2026-11-05 那一节讲的是**命令**：`break` 干什么、`print` 干什么。这一节讲**流程**：拿到一串红点之后，一次会话里到底按什么顺序做事。

把调试想成医生看诊，而不是背药名。病人（失败的测试）一进门只会说一句「这里疼」（`Should be true`）。医生的工作是固定的四步：

- **定位**：疼在哪个断言上？（红点行号）
- **取样**：把程序按在那一行动不了，读当时的所有变量；
- **假设**：读到的值与规格不一致，说明哪一步没做对；
- **验证**：改一处、重跑、看红点是否移动。

关键差别：`go test` 只完成第一步就结束了；Delve 让第二、三步变得便宜。所以工作流的价值不在于「会敲 `print`」，而在于**每次停下都要能排除掉一些可能性**。一次停下只确认了一件事，那一停就是浪费。

> 📌 **核心要点** 判断一次调试是否有效，看「停下之后排除了几个假设」。在失败断言行停下，一次能同时看到入参、返回值、被测对象状态，通常能排除掉一半假设；在实现函数入口停下，往往只能确认「参数传对了没有」。

2.2. 硬核机制

2.2.1. 第 0 步：红点是入口，先分类再动手

跑 `go test -count=1 ./internal/db/`（本机当前输出）：

```
    --- FAIL: TestKVBasic (0.00s)
        kv_test.go:36:
            Error:        Should be true
        kv_test.go:39:
            Error:        Should be true
        kv_test.go:48:
            Error:        Should be true
```

Listing 9: 三个红点，同一句失败信息

三条红点只有一句相同的话，但它们背后是三个互不相干的问题：

红点	断言	停下会看到	要回答的问题
36	`updated and err == nil`	`updated=false`，`err=nil`	Set 到底有没有给 `updated` 赋值
39	`string(val) == "v1" and ok and err == nil`	`val` 是 `[0,0]`，`ok=true`	`make` 之后有没有 `copy`

48	`updated and err == nil`（Del 命中已存在的键）	`updated=false`	`delete` 与「查存在性」谁在前

⚠️ **陷阱预警** 不要从第一个红点顺着改到最后一个。三条红点有三种根因，一次只追一个；追的过程中每改一次代码就重跑一次，红点会移动——移动本身就是信息（说明你改的正是它）。

2.2.2. 第 1 步：在失败的那条断言上断点

这是整个工作流里最重要的一条习惯：**断点打在断言行，不打在实现入口。**

```
    dlv test ./internal/db -- -test.run TestKVBasic
    (dlv) break kv_test.go:36
    (dlv) continue
    (dlv) print updated
    (dlv) print err
    (dlv) print kv.mem
```

Listing 10: 在第一条失败断言上停下，读三个值

本机实际输出（去掉源码上下文行）：

```
    Breakpoint 1 set at 0x69624b for github.com/zz-cpp/db-go/internal/
db.TestKVBasic() ./internal/db/kv_test.go:36
    > [Breakpoint 1] ... TestKVBasic() ./internal/db/kv_test.go:36 (hits
goroutine(8):1 total:1)
    false
    error nil
    map[string][]uint8 [
      "k1": [0,0],
    ]
```

Listing 11: 第 36 行停下时的 updated、err 与 mem

一次停下同时排除了三件事：

- `err` 是 `nil` → 失败**不是**错误路径造成的，不用去看 `ErrNotFound` 之类；
- `updated` 是 `false` 且 mem 里已经有 `k1` → 写入发生了，问题在 `updated` 的计算或压根没赋值；
- mem 里的值是 `[0,0]`（长度为 2 的零值） → 存入时没有拷贝内容，这已经预告了第 39 行会红。

这就是「一次停下排除多个假设」的样子。如果断点打在 `Set` 入口，你只能确认「key 和 val 传对了」，另外两条得再停一次。

2.2.3. 第 2 步：钻进实现，分清 args、locals 与返回值

当断言行提供的信息不够时，再进实现函数。`break db.(*KV).Set` 支持「包名简写」，Delve 会补全成完整符号：

```
break db.(*KV).Set
continue
args
next
next
print updated
locals
```

Listing 12: 在 Set 内部：先看参数，再走两步看返回值

本机实际输出（节选，省略地址与切片内容）：

```
> [Breakpoint 1] ... (*KV).Set() ./internal/db/kv.go:43
kv = ("*github.com/zz-cpp/db-go/internal/db.KV")(0x2ded5054d70)
key = []uint8 len: 2, cap: 2
val = []uint8 len: 2, cap: 2
updated = false
err = (unreadable could not resolve interface type)
> ... (*KV).Set() ./internal/db/kv.go:44
> ... (*KV).Set() ./internal/db/kv.go:45
false
(no locals)
```

Listing 13: args 给了参数，locals 却是空的

这里有三个必须记住的区分：

命令	给什么	本项目里的表现
`args`	函数参数（含接收者）	`kv` 、 `key` 、 `val` 都能读到
`locals`	只列**局部变量**，不含参数、不含命名返回值	`(no locals)`
`print`	任意表达式，包括命名返回值	`print updated` 得到 `false`

⚠️ **陷阱预警** `err` 显示 `(unreadable could not resolve interface type)` 不是程序 bug。函数入口处 `err` 还是零值 `error` 接口，Delve 无法从 DWARF 里解出接口的动态类型。用 `print err == nil` 这类**表达式**判断，而不是直接 `print err` 。

2.2.4. 第 3 步：条件断点，把重复调用过滤掉

`Get` 在测试里被调了三次（ `k1` 、 `xxx` 、 `xxx` ）。无条件断点会让你停三次、读三次相同的东西。条件断点的语法是「位置 + `if` + Go 表达式」：

```
break db.(*KV).Get if string(key) == "k1"
continue
args
print ok
next
print val
next
print val
```

Listing 14: 只在键是 k1 时停下，然后走两步看 make 的效果

本机实际输出：

```
> [Breakpoint 1] ... (*KV).Get() ./internal/db/kv.go:36
val = []uint8 len: 10044016, cap: 20162512119496
ok = true
> ... (*KV).Get() ./internal/db/kv.go:37
[]uint8 len: 0, cap: 0, nil
> ... (*KV).Get() ./internal/db/kv.go:38
[]uint8 len: 2, cap: 2, [0,0]
```

Listing 15: 同一变量的三次快照：垃圾值、nil、零字节

这三行快照是本节信息量最大的地方（`10044016` 这类数字每次运行都可能不同，它是栈上的残留，不是稳定的值）：

- 函数入口处 `val` 的 `len` 是 `10044016` —— **命名返回值在赋值前是栈上的垃圾**，Delve 照实读了出来。所以「入口处的返回值」永远不可信，别拿它下结论。
- 第一次 `next` 执行了 `val, ok = kv.mem[string(key)]` 之后的状态，`val` 变成 `nil`（未命中）或真实切片（命中）。
- 第二次 `next` 执行了 `val = make([]byte, len(val))`，`val` 变成 `len: 2` 的 `[0,0]` —— **长度对了、内容全零**，`make` 与 `copy` 的区别在这一行上肉眼可见。

2.2.5. 第 4 步：验证「顺序类」bug，用 next 走一遍副作用

第 48 行的红点属于顺序问题。这类 bug 的特征是：单个变量看都对，只有执行顺序错了。

```
break db.(*KV).Del
continue
args
next
next
print deleted
print kv.mem
next
print deleted
```

Listing 16: 在 Del 内部逐行走，观察 mem 与 deleted 的先后

本机实际输出（这次命中的是 `Del([]byte("xxx"))`，未命中路径）：

```
> [Breakpoint 1] ... (*KV).Del() ./internal/db/kv.go:49
key = []uint8 len: 3, cap: 3
deleted = false
err = error nil
> ... (*KV).Del() ./internal/db/kv.go:51
> ... (*KV).Del() ./internal/db/kv.go:52
false
map[string][]uint8 [
  "k1": [0,0],
]
> ... (*KV).Del() ./internal/db/kv.go:53
false
```

Listing 17: next 两次之后：删除已经发生，deleted 仍是 false

读法：`next` 两次后停在 `_, deleted = kv.mem[string(key)]` 之前，此时 `delete` 已经执行过；再 `next` 一次，`deleted` 变成 `false` —— 因为键已经被删掉，再查自然是「不存在」。**顺序反了**这个结论，是走出来的，不是猜出来的。

> ⚠️ **陷阱预警** `next` 与 `step` 的分工：`next` 把被调函数当一行执行完，`step` 钻进去。验证「当前函数的语句顺序」用 `next`；验证「被调函数内部发生了什么」用 `step` 或直接在被调函数上打断点。用错会让你在一堆标准库里迷路。

2.2.6. 第 5 步：调用栈、display 与 restart

`bt` 用来看「谁调了我」，测试场景下栈总是很短：

```
0  ... in github.com/zz-cpp/db-go/internal/db.TestKVBasic
   at ./internal/db/kv_test.go:48
1  ... in testing.tRunner
   at /usr/lib/go/src/testing/testing.go:2195
2  ... in testing.(*T).Run.gowrap1
   at /usr/lib/go/src/testing/testing.go:2260
3  ... in runtime.goexit
   at /usr/lib/go/src/runtime/asm_amd64.s:1264
```

Listing 18: bt 的四层：测试函数、tRunner、子测试包装、goroutine 出口

`testing.tRunner` 就是「每个测试跑在自己的 goroutine 里」的证据（`/usr/lib/go/src/testing/testing.go`）。

`display` 有两个本机实测的细节：

- 语法是 `display -a 表达式`，直接写 `display updated` 会报 `wrong arguments`（本机 dlv 1.27.2）；
- 它**只在断点命中时求值**，`next` 不会触发。所以「设了 display 却什么都没看到」是正常现象，需要 `continue` 到下一个断点。

改完代码后不用退出会话重来，`restart` 会重新编译测试二进制并重启（本机输出 `Process restarted with PID 56260`，PID 变了）。注意重启后停在**入口**，`print updated` 会报 `could not find symbol value for updated` —— 变量还没进入作用域，先 `continue`。

2.2.7. 第 6 步：把会话变成可重放的脚本

交互式敲一遍就够了，**值得复现的现场要写成脚本**。`--init` 喂命令文件，`--allow-non-terminal-interactive=true` 放行非终端：

```
dlv test ./internal/db \
  --allow-non-terminal-interactive=true \
  --init "$TMPDIR/dlv/s1.txt" \
  -- -test.run TestKVBasic
```

Listing 19: 本节所有输出都由这一条命令产生

`--` 之后的参数原样交给测试二进制；`-test.run` 的值是正则，所以 `-test.run 'TestKVBasic/case1'` 能选中子测试。脚本放会话临时目录，不进仓库。

2.2.8. 工作流全图

步	动作	看到什么就可以往下走	看到什么要回头
0	`go test -count=1`	拿到了行号	编译错误：先修编译
1	在失败断言处 `break`	读到了入参、返回值、被测对象	变量不在作用域：先 `next` 越过声明
2	判断「测试错」还是「实现错」	返回值与规格不符	`ok`、`err` 都符合规格：测试写错了
3	在实现里按函数名打断点	`args` 与预期一致	参数就不对：往上一层找
4	`next` 走顺序 / 条件断点过滤	定位到某一步没做或做反	走不通：换 `step` 或换断点
5	改一处、重跑	红点移动或消失	红点没动：改错了地方

2.3. 陷阱与易错

⚠ **陷阱预警 命名返回值在函数入口是垃圾值。**本机在 `Get` 入口读到 `val` 的 `len: 10044016`，那不是你的逻辑算出来的，是栈上残留。任何「入口处 print 返回值」得到的结论都作废。

⚠ **陷阱预警 `locals` 不含参数，也不含命名返回值。**`Set` 里 `updated`、`err` 都是命名返回值，`locals` 显示 `(no locals)` 并不表示「没有变量」，只表示「没有你要看的那个」。用 `args` 看参数、`print` 看返回值。

⚠ **陷阱预警 行号断点会漂。**改一行代码，`break kv_test.go:39` 可能指到别的语句上。稳定的写法是「函数名 + 条件」：`break db.(*KV).Get if string(key) == "k1"`；行号断点只用于一次性现场。

⚠️ **陷阱预警 在崩溃前一刻 print 不到东西。**想知道 panic 现场，不要设断点，直接 `dlv test` 后 `continue`：进程崩在 panic 点，`bt` 给出完整栈。这是唯一「不靠断点也能停下」的场合。

2.4. 关键点

- 断点位置决定信息量：**断言行** > **实现入口** > **任意行**。
- 一次停下至少排除一个假设；排除不了就说明断点位置选错了。
- `next` 看顺序，`step` 看内部，`bt` 看调用者，`args` 看入参，`print` 看一切表达式。
- 调试脚本是「现场的证据」，放会话临时目录，修完即可丢弃；结论要落进笔记（像本节这样）。
- 修完的红点会移动。**红点移动说明你改对了方向**，红点不动说明改错了地方。

📌 **核心要点** 这条工作流的终点不是「测试绿了」，而是「我能说清每个红点的根因，并且规格、测试、实现三处说法一致」。调试只是把这句话验证一遍的手段。

2.5. 变式检验

只给判断题：

- 第 36 行红点，如果断点打在 `Set` 入口而不是断言行，你还能同时排除「err 非 nil」这个假设吗？为什么？
- 第 39 行红点，用 `next` 从 `Get` 入口走三步，每一步 `val` 的 `len` 分别是多少？哪一步之后它才「可信」？
- 第 48 行红点，如果 `Del` 的实现是「先查后删」，`next` 走到 `delete` 之后 `print deleted` 应该是什么？
- 如果要确认 `Set` 同键同值时返回 `false`，断点应该设在测试的哪一行、条件是什么？

2.6. 同类归类

命令	停下时机	适合回答的问题	本节的例子
`break 行号`	执行该行之前	这一行的断言为什么失败	kv_test.go:36
`break 函数名 if 条件`	满足条件时进入函数	某一次调用的内部状态	`break db.(*KV).Get if string(key) == "k1"`
`next`	执行完当前行	语句顺序对不对	Del 的先删后查
`step`	进入被调函数	被调函数内部发生了什么	钻进 `Set` 看 `make`
`stepout`	执行完当前函数并返回	看返回值落到调用方	Set 的 `updated` 回到测试
`bt`	不移动	谁调用了当前函数	看到 `testing.tRunner`

`restart`	重新编译并回到入口	改完代码后重跑	PID 56260

2.7. Go 语言机制补充

- **测试就是一个可执行文件**：`go test` 在包模式下先编译出二进制再运行它（`go help testflag` 的 `-c` 只是把这个步骤单独暴露）。所以 `dlv test` 与 `dlv debug` 共用同一个调试内核，断点、变量、调用栈完全一致。
- **`dlv test` 自动关优化**：它给编译器加 `-gcflags=all=-N -l`。`-N` 让变量留在栈槽上，`-l` 保留函数边界——Delve 读变量位置和函数边界都依赖 DWARF 里的这两类信息。走 `dlv exec` 调试自己 `go test -c` 的产物时要手动加这两个标志，否则断点会漂、`locals` 会是空的。
- **命名返回值的初始化时机**：`func Get() (val []byte, ok bool, err error)` 的返回值在函数入口只是栈上的空间，尚未赋值。这不是陷阱，而是「零值」语义在调试器里的直接体现：**赋值之前，里面什么都不保证**。
- **`testing.tRunner` 与 goroutine**：`go test` 为每个测试起一个 goroutine 执行 `tRunner`（`/usr/lib/go/src/testing/testing.go`），所以 `bt` 的第二帧总是它；用 `t.Parallel` 时，先 `goroutines` 切到断点所在协程，否则看到的是别的测试的状态。
- **`--` 后面的参数**：`dlv test` 把 `--` 之后的全部内容交给测试二进制，`-test.run`、`-test.v`、`-test.bench` 都是 `go test` 传给测试二进制的标志名（加 `test.` 前缀是因为它们由 `flag` 包注册）。

2.8. 本次操作与当前状态

- 本轮实跑六个脚本（`$TMPDIR/dlv/s1.txt` 至 `s7.txt`），全部输出均已清洗 ANSI 颜色码，贴进本节的是节选而非全量。
- 期间发现 `internal/db/kv.go` 已被本人修改：`Get` 不再返回 error（`err` 恒为 nil）、`Set` 没有给 `updated` 赋值、`Del` 改成「先 delete 再查」。因此当前红点从 11-05 的 5 个降到 3 个：`kv_test.go` 第 36、39、48 行；第 42、45、51 行已通过。
- 未改动 `internal/db` 任何文件；未新增仓库文件。所有调试脚本、`go test -c` 产物都在会话临时目录。
- 本机环境：dlv 1.27.2、go 1.27.3，两者版本匹配。
- 待本人动手（按第 0 步的分类，一次只追一个）：① 36 行——`Set` 里 `updated` 的赋值；② 39 行——`Get` 的 `copy`；③ 48 行——`Del` 的「先查后删」。修完每一条都重跑 `go test -count=1 ./internal/db/`，看红点是否移动。
- 想复现本节任何一个现场：把 `#codelisting` 里的命令块存成文件，用第 6 步那条 `dlv test --init` 命令喂进去即可。

3. 在 Neovim 里调试 kv_test.go（nvim-dap + nvim-dap-go）｜问题 0101

3.1. 费曼直觉

Delve CLI 是「在一个终端里跟程序对话」；Neovim 调试是「把同一场对话搬进编辑器」：断点从 `break kv_test.go:36` 变成行号旁的一个红点，`print updated` 变成侧边栏里的一行变量，`bt` 变成 Stacktrace 面板。

两者底层是同一样东西。CLI 的 `dlv test` 让 Delve 自己跑测试；编辑器走的是 Delve 的 **DAP server**——`dlv dap -l 127.0.0.1:PORT`（本机 `~/.local/share/nvim/lazy/nvim-dap-go/lua/dap-go.lua` 第 66 行就是这么拼的）。编辑器通过 JSON 请求指挥 Delve，Delve 再指挥测试进程。

所以上一节的 CLI 工作流不会白学：**每个 Neovim 按键都能对应回一条 CLI 命令**。真正要新学的只有两件事——键位表，以及「调试整个包」和「调试单个测试」的入口不同。

> 📌 **核心要点** 本机不需要从零配置：Neovim 是 LazyVim，`dap.core` 与 `lang.go` 两个 extra 已经启用，`nvim-dap`、`nvim-dap-ui`、`nvim-dap-virtual-text`、`nvim-dap-go`、`neotest-golang` 全部已安装，`dlv` 在 `PATH` 里。你缺的不是插件，是键位和入口。

3.2. 硬核机制

3.2.1. 一、本机现状（已核实）

组件	状态	证据
Neovim	0.12.6	`nvim --version`
配置框架	LazyVim	`~/.config/nvim/lazyvim.json` 的 `extras`
调试前端	`nvim-dap` + `dap-ui` + 虚拟文本	`~/.local/share/nvim/lazy/` 下均有目录
Go 适配器	`nvim-dap-go`	LazyVim `lang.go` extra 引入
测试前端	`neotest-golang`，且 `dap_go_enabled = true`	`lang.go` 第 157 行
调试器	`dlv` 1.27.2（PATH 与 mason 各一份，同版本）	`dlv version`

3.2.2. 二、按一下键，它到底做了什么

`nvim-dap-go` 注册 adapter 时拼的命令（本机源码第 66 至 72 行）：

```
    local args = { "dap", "-l", "127.0.0.1:" .. config.delve.port }
    local delve_config = {
      type = "server",
      port = config.delve.port,
      executable = { command = config.delve.path, args = args, cwd =
config.delve.cwd },
    }
```

Listing 20: adapter 就是一条 `dlv dap -l 127.0.0.1:PORT` 命令

`debug_test()` 拼出的 launch 配置（同文件第 196 至 215 行）：

```
    local config = {
      type = "go",
      name = testname,
      request = "launch",
      mode = "test",
      program = testpath,
      args = { "-test.run", "^" .. testname .. "$" },
      buildFlags = build_flags,
      outputMode = "remote",
    }
```

Listing 21: 等价于 `dlv test ./internal/db -- -test.run '^TestKVBasic$'`

于是 CLI 与 Neovim 的对应关系是完整的：

CLI	DAP 请求	LazyVim 键位
`dlv test ./internal/db -- -test.run TestKVBasic`	`launch`（`mode=test`）	`<leader>td`
`break kv_test.go:36`	`setBreakpoints`	光标到该行 + `<leader>db`
`break db.(*KV).Get if string(key) == "k1"`	`setBreakpoints` + `condition`	`<leader>dB`（输入条件）
`continue`	`continue`	`<leader>dc`
`next`	`next`	`<leader>dO`
`step`	`stepIn`	`<leader>di`
`stepout`	`stepOut`	`<leader>do`
`print updated`	`evaluate`	`<leader>de` 或悬停 `<leader>dw`
`bt`	`stackTrace`	Stacktrace 面板；`<leader>dk` / `<leader>dj` 切帧
`restart`	`restart`	`<leader>dl`（Run Last）
`exit`	`disconnect` / `terminate`	`<leader>dt`

本机用同一套 DAP 路径 headless 实跑，停在 `kv_test.go` 第 36 行并求值：

```
FRAME=db.TestKVBasic@36
EVAL={ result = "false", type = "bool", variablesReference = 0 }
```

Listing 22: Neovim 侧的 evaluate 与 CLI 的 `print updated` 得到同一个值

3.2.3. 三、一次完整会话（按你现在的 kv_test.go）

步	按键	发生什么
1	打开 `internal/db/kv_test.go`	缓冲区进入 `go` filetype，dap-go 的配置生效
2	第 36 行 + `<leader>db`	行号旁出现断点标记
3	光标移进 `TestKVBasic` 函数体	neotest 才能认出「最近的测试」
4	`<leader>td`	启动 `dlv dap`，编译测试二进制，停在 36 行
5	`<leader>du`	打开 dap-ui：Scopes / Variables / Watches / Stacktrace / Breakpoints / REPL
6	`<leader>dO`	单步，Variables 里 `updated` 的值随之变化
7	`<leader>de`	对光标处表达式求值（可视模式选中也行）
8	`<leader>dr`	打开 REPL，可直接输入 `kv.mem` 这类表达式
9	`<leader>dt`	结束会话，测试进程被终止

不想经过 neotest 时，在测试函数里直接执行也行：

```
:lua require('dap-go').debug_test()
```

Listing 23: 它用 treesitter 找光标最近的测试函数名，再拼 `-test.run ^名字$`

`<leader>td` 是 neotest 的 debug 策略（LazyVim `test.core` 第 126 行：`run({strategy = "dap"})`），`dap_go_enabled = true` 让它落到 nvim-dap-go 上。两者最终走的是同一个 `debug_test`。

3.2.4. 四、条件断点：只在 `k1` 上停

`Get` 在测试里被调三次（`k1`、`xxx`、`xxx`）。在 `<leader>dB` 的提示里输入：

```
string(key) == "k1"
```

Listing 24: 条件断点：与 CLI 的 `if string(key) == "k1"` 完全一致，不需要转义引号

之后每次进入 `Get` 都会求值这个表达式，只有为真才停。这比「停三次再手动往下看」便宜得多。

3.2.5. 五、调试入口不止一个

`<leader>dc`（Run/Continue）第一次执行时会弹配置选单。本机已注册的 Go 配置（headless 实测列出）：

配置名	MODE	用途
`Debug`	launch	当前文件 `program = "${file}"`，适合 `main.go`
`Debug (Arguments)`	launch	启动时弹窗输入参数
`Debug Package`	launch	`program = "${fileDirname}"`，整个包
`Attach`	attach	选一个正在运行的进程挂上去
`Debug test`	test	当前文件按测试编译运行
`Debug test (go.mod)`	test	按相对模块路径跑整个目录的测试

调试单个测试一律用 `<leader>td`：它带 `-test.run ^名字$`，只跑你光标所在的那一个，不会把整个包跑一遍。

3.2.6. 六、真实项目里这套配置长什么样

VS Code 的 Go 扩展是另一个 DAP 客户端，`launch.json` 与本机 `dap.configurations.go` 字段一一对应（`mode: "test"`、`program`、`args` 里的 `-test.run`）。出处：`github.com/golang/vscode-go` 仓库的 `docs/debugging.md`。换句话说，**DAP 是标准，编辑器只是外壳**；学会一套字段，换编辑器不用重学。

如果你以后想脱离 LazyVim 的默认值（比如给测试加 `-race`），改的是 nvim-dap-go 的 `opts`，而不是写 `launch.json`：

```
{
  "leoluz/nvim-dap-go",
  opts = {
    delve = {
      path = "dlv",
      initialize_timeout_sec = 20,
    },
  },
}
```

Listing 25: 放在 `~/.config/nvim/lua/plugins/` 下的 override；`build_flags` 也可以在这里加

3.3. 陷阱与易错

⚠ **陷阱预警** `<leader>td` **提示** `no test found`：`debug_test` 用 treesitter 找「光标最近的测试函数」。光标停在文件头、import 区，或者文件不是 `_test.go`，都会找不到。把光标移进 `TestKVBasic` 的函数体再试。

⚠️ **陷阱预警 adapter 启动超时**：首次调试要先编译测试二进制，慢机器上可能撞上 `initialize_timeout_sec = 20` 。表现是「dap 报错但没有任何断点命中」。先手动跑一次 `go test ./internal/db/` 让编译缓存热起来。

⚠️ **陷阱预警 `dlv` 路径有两份。** `nvim-dap-go` 默认 `path = "dlv"` ，走 `PATH` （ `/home/zz/go/bin/dlv` ）；mason 也会装一份（ `~/.local/share/nvim/mason/bin/dlv` ）。本机两者都是 1.27.2，没问题；如果哪天升级了其中一个，记得两边对齐，否则会出现「CLI 能断、Neovim 断不准」这种最难查的偏差。

⚠️ **陷阱预警 不要用固定端口。**默认 `port = "${port}"` ，由 nvim-dap 每次分配空闲端口。手改成固定值后，上一次会话没退干净就会撞端口。

⚠️ **陷阱预警 会话结束不等于进程结束。** `<leader>dt` 是 terminate；如果只是关掉 dap-ui（ `<leader>du` ），会话还在跑。下一次 `<leader>td` 前用 `<leader>ds` 看一眼当前会话，避免「两个 dlv 同时占着一个测试」。

3.4. 关键点

- 你已有的三组键，覆盖了调试的 90%：**断点**（ `<leader>db` / `<leader>dB` ）、**跑与步进**（ `<leader>td` 、 `<leader>dc` 、 `<leader>dO` 、 `<leader>di` 、 `<leader>do` ）、**看值**（ `<leader>du` 、 `<leader>de` 、 `<leader>dr` ）。
- 调试测试用 `<leader>td` ，不要用 `<leader>dc` 再选 `Debug test` ——前者自动锁定光标所在的测试，后者按文件跑。
- 断点优先打在**断言**行（与 CLI 一节相同）：这里同时能看到入参、返回值、被测对象状态。
- Neovim 只是把 CLI 的每一步换成按键； `dlv` 的行为、 `-test.run` 的正则语义、变量在入口是垃圾值这些规律，一条都没变。

📌 **核心要点** 判断「会不会用 Neovim 调试」的标准不是记住快捷键，而是：给一个红点，你能说出**在哪一行下断点、断点条件是什么、打开哪个面板看哪个变量**。快捷键可以查，这三件事查不到。

3.5. 变式检验

只给判断题：

- 想在「 `updated` 为 false 时」停下，条件断点写什么？它和「在第 36 行停」有什么区别？
- `<leader>td` 与 `<leader>dc` 选 `Debug test` 跑的是同一个东西吗？差在哪个参数上？
- 为什么 Neovim 里不需要自己加 `-gcflags=all=-N -l` ，而 `dlv exec` 调试 `go test -c` 的产物时必须加？
- 调试结束后想再跑一次，用 `<leader>dl` 和重新 `<leader>td` 有什么区别？

3.6. 同类归类

方案	断点可视化	变量面板	取舍
`nvim-dap` + `nvim-dap-go`	有	`dap-ui`	本机已装，功能最全，需要一点键位记忆
`vim-delve`（sebdah）	有	简易	更早的方案，功能少，维护不活跃
`neovim` 内置 `:terminal` + `dlv`	无	无	永远是兜底；与 CLI 一节完全等价
VS Code + Go 扩展	有	有	图形化最顺手；同样是 DAP 客户端
`dlv dap` + 任意 DAP 客户端	取决于客户端	取决于客户端	协议是标准，编辑器可换

3.7. Go 语言机制补充

- **DAP 是三层结构**：编辑器（客户端）通过 JSON 与 adapter（`dlv dap`）通信，adapter 再控制目标进程（测试二进制）。所以「Neovim 调试」与「命令行调试」共用同一个 Delve 内核，差别只在传输层。
- **测试二进制与 `-test.run`**：`debug_test` 传的是 `-test.run ^TestKVBasic$`。`-test.run` 的值是正则（`go help testflag`），`^` 与 `$` 把名字钉死，避免误选 `TestKVBasicExtra` 这类前缀相同的测试。
- **treesitter 找测试函数**：`nvim-dap-go` 依赖 `ts.closest_test()` 从语法树上找最近的 `func TestXxx(t *testing.T)`，所以光标位置决定「调哪个测试」；这是纯文本搜索做不到的。
- **`outputMode = "remote"`**：测试进程的 stdout/stderr 由 Delve 通过 DAP 事件转发给编辑器，所以在 dap-ui 的 REPL 或输出面板里能看到 `--- FAIL` 之类的测试输出，而不是散落在某个终端。
- **优化关闭**：`dlv test` 与 `dlv dap`（`mode=test`）自己负责编译并关闭优化；只有手动 `go test -c` 再用 `dlv exec` 时才需要 `-gcflags=all=-N -l`（详见 2026-11-05 那一节的对照实验）。

3.8. 本次操作与当前状态

- 核实本机：Neovim 0.12.6、LazyVim（`dap.core` / `lang.go` / `test.core` 三个 extra）、`nvim-dap`、`nvim-dap-ui`、`nvim-dap-virtual-text`、`nvim-dap-go`、`neotest-golang`、`dlv` 1.27.2 全部就位。
- 读取本机插件源码确认行为：`nvim-dap-go/lua/dap-go.lua` 第 66 至 72 行（adapter 命令）、第 196 至 215 行（`debug_test` 的 launch 配置）；`LazyVim/lua/lazyvim/plugins/extras/dap/core.lua`（键位）、`extras/lang/go.lua` 第 141 与 157 行、`extras/test/core.lua` 第 126 行。
- headless 实跑两次：一次确认停在 `kv_test.go:36`（`thread=8`），一次用 DAP `evaluate` 求 `updated` 得到 `false`，与 CLI 的 `print updated` 一致。

- 未改动仓库文件，也未改动 Neovim 配置；探针脚本在会话临时目录（`dap_probe.lua`、`dap_probe2.lua`）。
- 待本人动手：打开 `internal/db/kv_test.go`，光标放第 36 行按 `<leader>db`，光标移进 `TestKVBasic` 按 `<leader>td`，然后用 `<leader>du` 看 Variables。三个红点（36 / 39 / 48）的追查顺序与上一节相同。

4. dlv test 命令的参数归谁解析：分界符与测试标志 | 问题 0101

4.1. 费曼直觉

这条命令里其实有**两个程序**在抢同一行参数：`dlv test` 是第一个，被它编译出来的**测试二进制**是第二个。

`--` 就是分界线：线左边的话说给 dlv 听，线右边的话原样转交给测试二进制。这和 `ssh host -- command`、`cargo run -- args`、`go test -args a b` 是同一种设计——「工具自己的参数」与「被工具运行的那个程序的参数」必须有个明确的分界，否则两边都想解析 `-v`、`-run` 这类通用名字。

知道了这一点，这条命令就能读成一句自然语言：

> 用 Delve 调试 `package` 这个包的测试；调试器自己的参数到此为止（`--`），后面这些是给测试二进制的——只跑名字匹配 `TestSomething` 的测试（`-test.run`），并且要详细输出（`-test.v`），其它参数也这样原样传（`-other-argument`）。

> 📌 **核心要点** `-other-argument` **不是真实参数名**，它是 Delve 帮助里的占位符，意思是「这里可以继续写任意多个测试二进制认识的参数」。把它当成人名里的「张三」来读就对了。

4.2. 硬核机制

4.2.1. 一、逐段拆解

片段	归谁解析	含义	依据
`dlv`	shell	调试器可执行文件	`command -v dlv`
`test`	dlv	子命令：编译测试二进制并开一个调试会话（自动关优化）	`dlv help test` 第一句
`[package]`	dlv	要调试的包，**可省略**；省略时用当前目录	`dlv help test`：By default Delve will debug the tests in the current directory
`--`	dlv 的 flag 解析器	结束 dlv 自己的参数解析，后面的内容原样传给测试二进制	`dlv help test`：Double-dashes `--` can be used to pass arguments to the test program
`-test.run TestSomething`	测试二进制	只跑名字匹配正则 `TestSomething` 的测试	`go help testflag` 的 `-run regexp` 段

`-test.v`	测试二进制	详细输出：每个测试打印 `=== RUN` / `--- PASS`	源码 `testing/testing.go` 第 460 行
`-other-argument`	测试二进制	占位：任意其它测试参数	示例本身

4.2.2. 二、`--` 之前：dlv 自己的部分

`dlv help test` 的原文（本机 1.27.2）：

```
    Compiles a test binary with optimizations disabled and begins a new debug
session.

    The test command allows you to begin a new debug session in the context
of your
    unit tests. By default Delve will debug the tests in the current
directory.
    Alternatively you can specify a package name, and Delve will debug the
tests in
    that package instead. Double-dashes `--` can be used to pass arguments to
the test program:

    dlv test [package] -- -test.run TestSomething -test.v -other-argument

    See also: 'go help testflag'.
```

Listing 26: `dlv help test` 的完整说明（节选）

三件事值得注意：

- 「**optimizations disabled**」**不是形容**：dlv 会自动给编译器加 `-gcflags=all=-N -l`，所以断点不会漂、变量读得出来。
- `[package]` **只有一个**：与 `go test ./...` 不同，Delve 一次只调试一个包。实测 `dlv test ./...` 直接报错（见下面的陷阱）。
- **包参数有三种写法**，本机都实测通过：

写法	例子	实测结果
省略	先 `cd internal/db`，再 `dlv test`	跑当前目录的测试
相对路径	`dlv test ./internal/db`	跑该目录的测试
导入路径	`dlv test github.com/zz-cpp/db-go/internal/db`	跑该包的测试

4.2.3. 三、`--` 之后：测试二进制的部分

`--` 之后的内容不经过 dlv 的 flag 解析，直接成为测试二进制的 `os.Args[1:]`。
测试二进制启动时，`testing.Init()` 会注册一批以 `test.` 开头的 flag（`/usr/lib/go/src/testing/testing.go` 第 438 行起）：

```
    func Init() {
      ...
      flag.Var(&chatty, "test.v", "verbose: print additional output")
      ...
      match = flag.String("test.run", "", "run only tests and examples
matching `regexp`")
      ...
    }
```

Listing 27: 测试二进制认识的 flag 由 testing 包自己注册（第 460、465 行）

这就是 `-test.` 前缀的来源：**flag 是测试二进制注册的，不是 dlv，也不是 go 命令。**

`-test.run` 的值是**正则**，不是全等匹配。 `go help testflag` 原文：

```
    -run regexp
        Run only those tests, examples, and fuzz tests matching the regular
        expression. For tests, the regular expression is split by unbracketed
        slash (/) characters into a sequence of regular expressions, and each
        part of a test's identifier must match the corresponding element in
        the sequence, if any.
```

Listing 28: 正则语义与按 `/` 切分的子测试匹配规则

所以 `-test.run TestSomething` 里的 `TestSomething` 完全可以直接写成 `TestKV.*` 。本机实测（`-- -test.run 'TestKV.*' -test.v`）：

```
    === RUN   TestKVBasic
              Error Trace:  .../internal/db/kv_test.go:36
              Error:        Should be true
    --- FAIL: TestKVBasic (0.00s)
    FAIL
```

Listing 29: 正则选中了 TestKVBasic； `-test.v` 让 RUN/FAIL 行出现

如果没有 `-test.v` ，同样的运行只会输出失败摘要，看不到 `=== RUN` 这一行。

4.2.4. 四、为什么必须有 `--` ：反面实测

漏掉 `--` ，把测试参数直接写在 dlv 后面：

```
    dlv test ./internal/db -test.run TestKVBasic
```

Listing 30: 错误写法：测试参数没有分界

本机输出：

```
    with multiple packages, -o must refer to a directory or /dev/null
    exit status 1
```

Listing 31: 参数被当成了额外的包，编译阶段就失败

读法：dlv 把 `-test.run` 之后的词当成了**包列表**的一部分，转手交给 go 命令；go 命令发现「多个包 + 单个 `-o`」就报这条错。**测试参数一个都没传到测试二进制**，也根本没进入调试。

4.2.5. 五、为什么是 `-test.v` 而不是 `-v`

`go help testflag` 第 228 至 230 行的原文：

```
    Each of these flags is also recognized with an optional 'test.' prefix,
    as in -test.v. When invoking the generated test binary (the result of
    'go test -c') directly, however, the prefix is mandatory.
```

Listing 32: 带不带前缀，取决于「谁在启动测试二进制」

机制：`go test` 会**重写或移除**它认识的 flag，再启动二进制，所以人写 `go test -v` 也能工作；而 dlv（以及 `go test -c` 的产物）是**直接启动**测试二进制的，没有中间人帮你转写，前缀就变成强制的。

本机实测 `-- -v`：

```
    flag provided but not defined: -v
```

Listing 33: 测试二进制的 flag 包里没有 `-v` 这个名字

4.2.6. 六、`-other-argument` 能换成什么

它可以是**任何测试二进制认识的 flag**，常见的有：

参数	作用	本项目里的用途
`-test.timeout 30s`	整体超时	防止死循环的测试卡住调试会话
`-test.count=1`	禁用结果缓存、重复跑 N 次	与 `go test -count=1` 等价
`-test.bench .`	跑基准测试	给 `BenchmarkXxx` 下断点前先用它进入循环
`-test.cpuprofile`	写 CPU profile	调试与性能分析同时做
`-test.v`	详细输出	让 `=== RUN` 行出现在调试器输出里

它也可以是**非 flag 的参数**，这些会留在测试二进制的 `os.Args` 里，供测试代码自己读取（`go test -args` 的等价物）。本项目没有读 `os.Args` 的测试，所以这条暂时用不上。

4.3. 陷阱与易错

> ⚠️ **陷阱预警 漏掉 `--`**：错误信息看起来和调试毫无关系（`with multiple packages, -o must refer to a directory or /dev/null`），因为它其实是**编译阶段**的报错。看到这条，先检查 `--` 在不在。

⚠️ **陷阱预警 `dlv test ./...` 不支持。**本机实测报的是同一条 `with multiple packages` 错误。要调试多个包，只能一个一个来，或用 `dlv test` 加单个目录。

⚠️ **陷阱预警 `-test.run` 是正则，不是名字全等。** `-test.run TestKV` 会同时选中 `TestKVBasic` 、 `TestKVExtra` 。要钉死一个测试就写 `-test.run '^TestKVBasic$'` ——这正是 Neovim 的 `nvim-dap-go` 在 `debug_test` 里干的事（上一节源码第 202 行）。

⚠️ **陷阱预警 `-test.v` 的输出属于测试二进制，不属于 dlv。**在 dlv 会话里看不到 `=== RUN` ，往往是因为忘了 `-test.v` ，而不是 dlv 吞了输出；同理 `--test.timeout` 这种拼写错误会让测试二进制直接报 flag 未定义。

⚠️ **陷阱预警 参数在会话启动后就固定了。**想换 `-test.run` 的值，得退出重开（或在新会话里用不同的 launch 配置）。Delve 的 `restart` 不会重新解析命令行参数。

4.4. 关键点

- 一条命令两套参数： `--` **左边归 dlv，右边归测试二进制。**
- 右边所有 flag 都要带 `test.` 前缀，因为它们是 `testing.Init()` 注册的，不是 dlv 注册的。
- `[package]` 是单个包，可用相对路径或导入路径；省略时用当前目录。
- `-test.run` 收正则， `-test.v` 是布尔开关， `-other-argument` 是「还能写别的」的占位。

📌 **核心要点** 以后看到任何 `X -- Y` 形式的命令，先问一句「Y 是给哪个程序解析的」。 `dlv test` 的 `--` 、 `go test -args` 、 `cargo run --` 、 `ssh host --` 都是同一个模式；分界符之后的内容，语义由**被运行的那个程序**决定。

4.5. 变式检验

只给判断题：

- 只想跑 `TestKVBasic` 下的子测试 `case1` ， `-test.run` 的值怎么写？(提示： `/` 的切分规则)
- 把 `-test.v` 换成 `-v` 会怎样？为什么 `go test -v` 却没问题？
- `dlv test ./internal/db -- -test.run TestKVBasic -test.timeout 10s` 里，哪个参数由 dlv 解析？
- 如果想让测试代码读到自己的命令行参数（比如一个文件路径），应该写在 `--` 的哪一侧？

4.6. 同类归类

命令	分界符	之后归谁	本项目的等价写法
`dlv test pkg -- flags`	`--`	测试二进制	`-test.run` / `-test.v`
`go test -args a b`	`-args`	测试二进制	本项目未用到
`go run main.go -- a`	`--`	被运行的程序	无
`cargo run -- args`	`--`	被运行的程序	无
`ssh host -- cmd`	`--`	远端 shell	无
`go test -c` 的产物直接跑	无分界符	测试二进制只认 `test.` 前缀	调试脚本里常见

4.7. Go 语言机制补充

- **`testing.Init()` 注册 flag**：测试二进制在 `main` 开头调用 `testing.Init()`，它用标准库 `flag` 包注册 `test.v`、`test.run`、`test.timeout` 等（`/usr/lib/go/src/testing/testing.go` 第 438 至 470 行）。所以「`-test.` 前缀」这件事的根源在 `testing` 包，而不在 dlv。
- **`go test` 的转写**：`go help testflag` 第 232 行起说明，`go test` 会「rewrites or removes recognized flags」，把 `-v` 转成 `-test.v` 再传给二进制；`go test -c` 的产物没有这层转写，前缀因此强制。
- **正则与子测试**：`-test.run` 的值按**未转义的** `/` 切分，逐段匹配父测试与子测试名；父测试即使没有匹配的子测试也会被执行（因为要先跑父测试才能发现子测试）。所以 `TestKVBasic/case1` 能选中子测试。
- **`dlv test` 的编译方式**：它先 `go test -c` 生成测试二进制，编译时加 `-gcflags=all=-N -l`（关闭优化与内联），再启动调试会话。这也是为什么它「慢一点但断点准」。
- **`--` 的解析者**：dlv 用 cobra 解析命令行，`--` 之后的 token 不再当 flag 处理，而是作为位置参数交给子命令；`dlv test` 再把它们原样塞进测试二进制的 `os.Args`。

4.8. 本次操作与当前状态

- 本机实跑六条命令，结论均有输出为证：`dlv help test`、`go help testflag`（`-run regexp` 段与 `test.` 前缀段）、`-- -test.run 'TestKV.*' -test.v` 成功跑出 `=== RUN`、漏 `--` 报 `with multiple packages`、`-- -v` 报 `flag provided but not defined`、`dlv test ./...` 同样报多包错误；包参数的三种写法（省略、相对路径、导入路径）均通过。
- 读取 `/usr/lib/go/src/testing/testing.go` 第 438、460、465 行确认 `test.v` 与 `test.run` 的注册位置。
- 未改动 `internal/db` 任何文件；调试 init 脚本在会话临时目录。
- 待本人动手：无。这条命令的用法可以直接套到 0101 的三个红点上，例如只追第 39 行时用 `dlv test ./internal/db -- -test.run '^TestKVBasic$'`。

5. 为什么 delve 的 `list` 显示 `no source available`：PC 落在了没有行表的地址上｜问题 0101

5.1. 费曼直觉

`list` 想干的事是「把当前停在的那行代码显示出来」。它需要三样东西同时成立：

- **当前 PC**：调试器知道程序停在哪（这个总有）；
- **行表**：一张「地址 → 文件:行号」的映射表，由编译器写进二进制的 DWARF 段；
- **源文件**：那个 `文件:行号` 指向的 `.go` 文件此刻还在磁盘上。

`no source available` 的意思是：**这张地图上没有这个坐标**。它不是「你的代码被删了」，也不是「项目坏了」，而是「程序停在了你写的 Go 代码还没开始执行的地方」。

本机复现（`dlv test` 一启动就 `list`）：

```
(dlv) list
Stopped at: 0x7f2c9e22e400
=>   1: no source available
(dlv) bt
0  0x00007f2c9e22e400 in ???
   at ?:-1
```

Listing 34: 三个信号：0x7f 开头的地址、???、? 加 –1 的源码位置

三个信号合起来只说明一件事：**进程刚被创建，还停在动态链接器里，Go 的运行时尚未接管**。

> 📌 **核心要点** 读 `no source available` 的第一反应不应该是「调试器坏了」，而应该是「我现在停在哪」。`bt` 的输出就是答案：只有一帧、函数名是 `???`、位置是 `?:-1`，说明这一帧根本不属于你的程序。

5.2. 硬核机制

5.2.1. 一、为什么启动瞬间会停在「没有源码」的地方

Linux 执行一个**动态链接**的程序时，内核并不会直接把控制权交给程序的入口，而是先执行 ELF 头里 `INTERP` 指定的解释器（`/lib64/ld-linux-x86-64.so.2`）。因此进程刚起来时，PC 落在动态链接器的映射区——那是一块 `mmap` 出来的高地址，**没有任何 Go 的 DWARF 条目**。

本项目的测试二进制正是动态链接的（`go test -c` 产物，本机实测）：

```
ELF 64-bit LSB executable, x86-64, dynamically linked,
  interpreter /lib64/ld-linux-x86-64.so.2, with debug_info, not stripped
```

Listing 35: `file` 的输出：动态链接、带调试信息、未 strip

构建信息里也能看到根因（`go version -m` 输出节选）：

```
    build -buildmode=exe
    build CGO_ENABLED=1
```

Listing 36: 开启了 cgo 的构建会链上系统 C 库，因此需要动态链接器

于是地址分布可以这样读：

地址范围	典型内容	`list` 的表现
`0x4xxxxx` 附近	Go 代码与 Go 运行时（本机 `go test` 的断点都在 `0x69624b` 这类地址）	正常显示 `.go` 源码
`0x7f...` 高地址	共享库、`ld.so`、VDSO、线程栈	极可能是 `???` / `no source available`
任意地址	Go 汇编函数（`.s`）	**仍然有源码**，见下面的实测

5.2.2. 二、对照实测：同一个 `list`，换个时机就正常

先设断点再 `continue`，命中后 `list`：

```
    (dlv) break kv_test.go:36
    (dlv) continue
    > [Breakpoint 1] ... TestKVBasic() ./internal/db/kv_test.go:36 (PC:
0x69624b)
    (dlv) list
        31:   err := kv.Open()
        32:   assert.Nil(t, err)
        33:   defer kv.Close()
        35:   updated, err := kv.Set([]byte("k1"), []byte("v1"))
    =>  36:   assert.True(t, updated && err == nil)
```

Listing 37: 断点命中后，PC 落在 Go 代码里，`list` 立刻有内容

两次 `list` 的差别只有一件事：**PC 换了一个地址**。所以 `no source available` 从来不是「命令用错了」，而是「此刻没有代码可列」。

5.2.3. 三、Go 的汇编函数仍然有源码

很多人会把 `no source available` 归因于「停在了汇编里」。这在 Go 里不成立——Go 的 `.s` 文件参与行表，`list` 会直接显示汇编源文件：

```
    (dlv) break runtime.futex
    (dlv) continue
    > [Breakpoint 1] runtime.futex() /usr/lib/go/src/runtime/
sys_linux_amd64.s:568
    (dlv) list
        567:  TEXT runtime·futex(SB),NOSPLIT,$0
    =>  568:    MOVQ  addr+0(FP), DI
        569:    MOVL  op+8(FP), SI
```

Listing 38: 停在 `runtime.futex` 时，`list` 显示的是 `sys_linux_amd64.s`

真正没有行表的是「不属于这个二进制的东西」：动态链接器、VDSO、内核注入的 trampoline、以及被 `-ldflags="-s -w"` 剥掉的调试信息。

> ⚠️ **陷阱预警** `runtime.futex` 那次运行还打印了 `Warning: debugging optimized function`。这是「没关优化」的信号（本项目用 `dlv exec` 手动加载未关优化的二进制才出现）。它和 `no source available` 是两件事：优化会让变量读不准，但不会让源码消失。

5.2.4. 四、按这个思路排查的顺序

步	命令	看什么
1	`bt`	只有一帧且是 `???` → 停在启动阶段；有多帧 → 往上看哪一帧有 `.go`
2	`goroutines`	启动早期几乎没有 Go 协程；有正常协程说明程序已经跑起来了
3	`continue`	跑到断点或程序结束。启动瞬间的 `no source` 到这一步就消失
4	`break main.main` 再 `continue`	想直接停在业务代码入口时用这个
5	`frame 1` / `up`	当前帧没源码但上层有（例如停在运行时函数里）时，切上去看
6	`disassemble -l`	仍然想知道停在执行什么指令时，看反汇编

5.2.5. 五、如果 `continue` 之后还是 `no source available`

那就不是「启动阶段」了，按可能性从高到低查：

原因	怎么确认	怎么解决
二进制被 strip	`file` 里没有 `with debug_info`，或 `go tool nm` 看不到符号	去掉 `-ldflags="-s -w"` 重新编译
源码路径对不上	dlv 提示 `could not find file ...`（注意：这是另一条消息）	用 `config substitute-path` 映射路径
停在运行时或 VDSO	`bt` 的帧都是 `runtime.*` 或 `???`	`frame 1` 往上切，或用条件断点直接停在业务代码
断点位置被优化内联	`Warning: debugging optimized function`	用 `dlv test`（自带 `-N -l`）而不是 `dlv exec` 未关优化的产物

5.3. 陷阱与易错

⚠️ **陷阱预警** `list` **不会自动帮你找到有源码的那一帧**。它只看当前帧。想看上层的代码，必须先 `frame 1`（或 `up`）切换，再 `list`。

⚠️ **陷阱预警 把** `no source available` **当成「源码丢了」去重建环境**，是这条消息最常见的误判。先 `bt`：如果只有 `???` 一帧，你的代码从来没被错过。

⚠️ **陷阱预警 用** `dlv attach` **调试阻塞中的进程，第一眼几乎总是** `no source`。被 attach 的进程通常正卡在系统调用里（`futex`、`epoll_wait`、`nanosleep`），PC 在运行时或内核返回点。`continue` 一次让它走到下一个断点即可。（本机 `ptrace_scope` 限制导致 attach 实验无法进行，这条依据的是通用行为，未在本机实测。）

⚠️ **陷阱预警 Delve 在启动时默认「停住」**，不是「跑起来」。`dlv test` 后直接 `list` 看到的就是这个停住点。要么先 `break` 再 `continue`，要么接受它显示 `no source available`。

5.4. 关键点

- `no source available` = **当前 PC 地址在行表里查不到**，与源码是否存在无关。
- 启动瞬间停在 `ld.so`（地址 `0x7f...`、函数名 `???`、位置 `?:-1`）是最常见原因；`continue` 到断点即恢复。
- Go 的 `.s` 汇编**有**源码，别把「汇编」当理由。
- 真正的病态情况只有两种：二进制被 strip、源码路径不匹配；两者都有别的提示或可查证据。
- 排查顺序固定：`bt` → `goroutines` → `continue` → `frame 1` → `disassemble -l`。

📌 **核心要点** 调试时判断「我在哪」的成本极低：一条 `bt`。养成「先 `bt` 再 `list`」的习惯，这类问题就不会再花时间。

5.5. 变式检验

只给判断题：

- `list` 显示 `no source available` 且 `bt` 有两帧，第二帧是 `db.TestKVBasic at kv_test.go:39` —— 怎么让 `list` 显示源码？
- 项目用 `go build -ldflags="-s -w"` 打包后调试，`list` 会怎样？`bt` 还能显示函数名吗？
- 为什么 `runtime.futex` 有源码，而 `ld.so` 里的地址没有？
- `dlv test` 启动后直接 `continue`（不设断点），会发生什么？`list` 还有意义吗？

5.6. 同类归类

场景	同样的现象	本质
GDB 的 `list`	`No line number information available`	同一个原因：PC 没有行表
VS Code / nvim-dap	源码视图空白或跳到反汇编	DAP 的 `stackTrace` 返回 `source: null`
`go tool objdump`	能看到反汇编但看不到源码	行表缺失时的另一条观察途径
core dump 分析	停在信号处理帧时无源码	信号 trampoline 不属于用户代码

5.7. Go 语言机制补充

- **ELF 入口与 INTERP**：动态链接的可执行文件在 ELF 头里有 `INTERP` 段，内核先加载解释器并把 PC 设到它那里。Go 的 `runtime.rt0_go` 只有在运行时接管之后才会被执行。
- **行表来自 DWARF**：编译器为每条指令生成 `DW_LNE_set_address` 序列（`.debug_line` 段），Delve 用它在「地址 → 文件:行号」之间双向查找。没有这段信息，`list` 只能给 `no source available`。
- **Go 的 `.s` 也参与行表**：汇编器为汇编函数生成行条目，指向 `.s` 文件，所以 `runtime.futex` 能显示 `sys_linux_amd64.s:568`。
- **`-s -w` 的代价**：`-s` 去掉符号表与 DWARF，`-w` 去掉 DWARF 调试信息；两者一起用会让体积变小，但也让调试器失去源码映射。
- **`substitute-path`**：Delve 的 `config substitute-path <from> <to>` 用于「编译机器路径 ≠ 本机路径」的情况，属于「行表存在但文件找不到」，与本节讨论的「行表不存在」是两类问题。

5.8. 本次操作与当前状态

- 本机复现三次：① `dlv test ./internal/db` 启动后立即 `list`，得到 `Stopped at: 0x7f2c9e22e400` 与 `??? at ?:-1`（与你的现象同构）；② 断点命中后 `list` 正常显示 `kv_test.go` 第 31 至 41 行；③ `dlv exec` 一个 `time.Sleep` 程序并断在 `runtime.futex`，`list` 显示 `sys_linux_amd64.s:568`。
- 用 `file` 与 `go version -m` 确认测试二进制是动态链接（`interpreter /lib64/ld-linux-x86-64.so.2`，`CGO_ENABLED=1`）。
- `dlv attach` 实验因本机 `ptrace_scope` 限制失败（`Could not attach ... try writing 0 to /proc/sys/kernel/yama/ptrace_scope`），该条结论只按通用行为说明，未在本机验证。
- 未改动 `internal/db` 任何文件；演示程序与测试二进制在会话临时目录。
- 待本人动手：在 `dlv test ./internal/db` 里先 `break kv_test.go:36` 再 `continue`，然后 `list`；如果仍有 `no source available`，把 `bt` 与 `goroutines` 的输出发我。

6. 怎么按函数名设断点：`break` 的 locspec 与把函数名参数化 ｜问题 0101

6.1. 费曼直觉

`break` 的第一个位置参数不是「断点编号」，而是一个**位置描述符**（locspec）。函数名只是 locspec 的一种写法；`文件名:行号`、`行号`、`+3`、`*0x地址`、`/正则/` 都是它的兄弟。

所以「把 `Set` 作为参数传给 `break`」这件事，本质上是：

> 把 `Set` 写成**足够精确的 locspec**，让 Delve 能唯一定位到一个代码位置。

`help break` 的原话（会话内执行 `help break` 的输出）：

```
    break [name] [locspec] [if <condition>]

    Locspec is a location specifier in the form of:

      * *<address> Specifies the location of memory address address.
      * <filename>:<line> Specifies the line in filename.
      * <line> Specifies the line in the current file
      * +<offset> Specifies the line offset lines after the current one
      * -<offset> Specifies the line offset lines before the current one
      * <function>[:<line>] Specifies the line inside function.
          The full syntax for function is <package>.(*<receiver
type>).<function name> however
          the only required element is the function name, everything else can
be omitted as
          long as the expression remains unambiguous.
      * /<regex>/ Specifies the location of all the functions matching regex

    If locspec is omitted a breakpoint will be set on the current line.
```

Listing 39: `help break` 的位置描述符清单

> 📌 **核心要点** 注意 `<function>[:<line>]` 后面的那句话：**只有函数名是必需的**，包名、接收者类型都能省——前提是「expression remains unambiguous」。这正是为什么 `break Set` 在本项目里会失败，而 `break db.(*KV).Set` 会成功。

6.2. 硬核机制

6.2.1. 一、函数名的五种写法（全部本机实测）

写法	命令	实测结果
简写包名	`break db.(*KV).Set`	kv.go:48
完整导入路径	`break github.com/zz-cpp/db-go/internal/db.(*KV).Set`	同上
省略星号	`break db.KV.Set`	也能解析到 kv.go:48
只写函数名	`break Set`	❌ `Location "Set" ambiguous`

正则	`break /db\.\(\*KV\)\.Set/`	kv.go:48，只设一个

「只写函数名」失败时的完整报错（节选）：

```
    Location "Set" ambiguous: github.com/zz-cpp/db-go/internal/db.(*KV).Set,
    internal/abi.(*IntArgRegBitmap).Set, crypto/internal/fips140/nistec.
(*P521Point).Set,
    testing.(*durationOrCountFlag).Set, crypto/internal/fips140/edwards25519.
(*Point).Set
```

Listing 40: Delve 把候选列出来，让你自己收窄——这就是「无歧义」的含义

⚠️ **陷阱预警** `db.KV.Set` 能解析成功是 Delve 的容错，不是推荐写法。官方语法是 `<package>.(*<receiver type>).<function name>`；你的 `Set` 就是指针接收者（`func (kv *KV) Set`），规范写法是 `db.(*KV).Set`。如果哪天同一个类型上同时有值接收者和指针接收者的同名方法，省略星号就可能解析到另一个。

6.2.2. 二、`:行偏移`：进函数后第几行

`<function>:<line>` 里的 `<line>` 是**函数体内的行偏移**，不是文件行号。本机实测：

```
    (dlv) break db.(*KV).Set
    Breakpoint 1 set at ... (*KV).Set() ./internal/db/kv.go:48
    (dlv) break db.(*KV).Set:2
    Breakpoint 2 set at ... (*KV).Set() ./internal/db/kv.go:50
```

Listing 41: 不带偏移停在函数声明行；`:2` 停在函数体内第二行

对照当前的 `kv.go`：第 48 行是 `func (kv *KV) Set(...)` 的声明，第 49 行是 `kv.mem[...] = make(...)`，第 50 行是 `updated = bytes.Equal(...)`。所以 `:2` 正好落在「算 updated」那一行——**想直接停在某个关键语句上，用偏移比数文件行号稳**（改代码后行号会漂，偏移不会）。

6.2.3. 三、命名断点：给断点起名，之后用名字操作

`help break` 里还有一段：

```
    If you would like to assign a name to the breakpoint you can do so with
the form:

      break mybpname main.go:4

    Finally, you can assign a condition to the newly created breakpoint by
using the 'if'
    postfix form, like so:

      break main.go:55 if i == 5

    Alternatively you can set a condition on a breakpoint after created by
using the 'on' command.
```

Listing 42: 命名断点与两种加条件的方式

本机实测（先清掉同名位置的旧断点，否则会报 `Breakpoint exists`）：

```
    (dlv) break setbp db.(*KV).Set
    Breakpoint setbp set at 0x695cd3 for ... (*KV).Set() ./internal/db/
kv.go:48
    (dlv) on setbp string(key) == "k1"
```

Listing 43: 给断点起名 `setbp`，之后用 `on` 补条件

命名之后可以做的操作：`on setbp <条件>`（加条件）、`condition`、`clear setbp`（按名字删除）、在脚本里反复引用同一个断点。这就是「函数名做成参数」最实用的形态。

6.2.4. 四、一次设一批：正则 locspec

`break /Set/` 会匹配**所有**名字含 `Set` 的函数。本机实测：一次设了 **288 个**断点（`bytes.(*asciiSet).contains`、`crypto/internal/fips140/...SetBytes` 全都中枪）。

要收窄就写完整正则：

```
    (dlv) break /db\.\(\*KV\)\.Set/
    Breakpoint 1 set at 0x695cd3 for ... (*KV).Set() ./internal/db/kv.go:48
```

Listing 44: 正则里转义 `.` `(` `)`，只命中本项目的方法

6.2.5. 五、真正把函数名做成「参数」：脚本与 shell 函数

Delve 的 `--init` 接受一个命令文件，所以函数名可以当成变量填进去。下面是一个 fish 函数（本项目日常最省事的形态）：

```
    function dset
        printf 'break %s\ncontinue\n' $argv[1] > /tmp/dset.init
        dlv test ./internal/db --init /tmp/dset.init -- $argv[2..-1]
    end
```

Listing 45: `dset 'db.(*KV).Set' -test.run '^TestKVBasic$'` 即停在 Set 入口

bash 版本同理：

```
dset() {
  local fn="$1"; shift
  printf 'break %s\ncontinue\n' "$fn" > /tmp/dset.init
  dlv test ./internal/db --init /tmp/dset.init -- "$@"
}
```

Listing 46: 函数名是第一个参数，其余参数原样交给测试二进制

⚠️ **陷阱预警** 不要拿函数名直接拼文件名。`db.(*KV).Set` 里的 `(`、`)`、`*` 在 shell 里会被展开或需要转义。把 init 文件固定成一个路径（如 `/tmp/dset.init`），函数名只作为**文件内容**传进去，就不会有这个问题。

6.2.6. 六、Neovim 里没有「函数名断点」这回事

DAP 的 `setBreakpoints` 请求只认**行号**与**条件**，没有「函数名」字段。所以 Neovim 里想「按 `Set` 下断点」，实际路径是**先找出函数所在行，再把行号交给 nvim-dap**：

```
vim.cmd('edit internal/db/kv.go')
local line = vim.fn.search([[func (kv \*KV) Set]])
require('dap').set_breakpoint('string(key) == "k1"')  -- 在当前行（即 line）设条件断点
```

Listing 47: 本机 headless 实测：LINE=48，断点落在 48 行且带条件

实测返回值：

```
LINE=48
BPS={ { { buf = 1, condition = 'string(key) == "k1"', line = 48 } } }
```

Listing 48: `dap.breakpoints.get()` 的实际内容

也可以接在 LazyVim 的 `<leader>dB` 后面，把「光标处条件断点」改成「函数入口条件断点」：先用 `vim.fn.search` 把光标移到函数行，再触发 `<leader>dB`。

📌 **核心要点** CLI 的 `break <函数名>` 是 Delve 自己把符号解析成地址；Neovim 走 DAP，编辑器只发「文件 + 行号」。两者能力不同：**CLI 能按函数名，DAP 只能按行号**。想在编辑器里按函数名，就得自己做「函数名到行号」的转换（上面那行 `vim.fn.search`）。

6.3. 陷阱与易错

⚠️ **陷阱预警** `break Set` **报 ambiguous 不是你写错了函数名**，是这个二进制里 `Set` 有多处。Delve 会把候选全列出来，照着列表补上包名或接收者即可。

⚠️ **陷阱预警 同名位置不能重复设断点。**先 `break db.(*KV).Set` 再 `break setbp db.(*KV).Set` 会报 `Breakpoint exists at ...:48`，命名断点建不起来。要么先 `clear 1`，要么一开始就用命名形式。

⚠️ **陷阱预警 `init` 函数不能按函数名设断点。**`help break` 明确说：`For setting a breakpoint on an init function (ex: main.init), the <filename>:<line> syntax should be used`。一个包里可能有多个 `init`，函数名不足以区分。

⚠️ **陷阱预警 正则放在 `/` 之间，且要自己转义 `.`、`(`、`)`。**`/Set/` 会命中标准库 288 个函数；`/db\.\(\*KV\)\.Set/` 才只命中你的方法。设完记得 `clearall` 或用命名断点逐个清。

6.4. 关键点

- `break` 的第一个位置参数是 locspec；函数名的规范写法是 `<package>.(*<receiver type>).<function name>`，可省略到「无歧义」为止。
- `:N` 是**函数内行偏移**，不是文件行号；改代码后它比行号稳定。
- 想反复用同一个断点：给它起名（`break setbp db.(*KV).Set`），然后用 `on setbp <条件>` / `clear setbp`。
- 想一次设一批：用 `/正则/`，但一定要收窄。
- 编辑器（DAP）只认行号；「按函数名」要自己转成行号。

📌 **核心要点** 判断自己会不会用 `break` 的标准不是记住语法，而是：给你一个方法名，你能说出**它的完整 locspec 长什么样、用不用 `*`、要不要加偏移、要不要加条件**。本项目的答案是：`break db.(*KV).Set` 停入口，`break db.(*KV).Set:2` 停在算 `updated` 的那一行，再加 `if` 过滤。

6.5. 变式检验

只给判断题：

- `func (kv *KV) Get(...)` 与 `func (kv KV) Get(...)` 同时存在时，`break db.KV.Get` 会解析到哪一个？为什么？
- 想让断点停在 `kv.go` 第 49 行（`make` 那一行），用 `break db.(*KV).Set:1` 还是 `break db.(*KV).Set:2`？
- `break /KV/` 与 `break /db\.\(\*KV\)/` 分别会设多少个断点？
- 在 Neovim 里想对 `Del` 下条件断点，`vim.fn.search` 的模式该怎么写？

6.6. 同类归类

场景	写法	依据

按函数名	`break <包>.(*<接收者>).<函数>`	`help break` 的 locspec
按名字批量	`break /正则/`	同上
按文件行	`break kv.go:49`	同上
相对当前行	`break +3` / `break -2`	同上
按地址	`break *0x695cd3`	同上
DAP / 编辑器	只有行号 + 条件	`setBreakpoints` 请求的字段

6.7. Go 语言机制补充

- **方法符号名怎么来的**：编译器把方法编译成带接收者的符号，格式是 `<pkg>.(*<Type>).<Method>`（指针接收者带 `(*T)`，值接收者带 `(T)`）。Delve 的 locspec 就是这个符号名的宽松形式。
- **为什么要写 `(*KV)`**：Go 允许对可寻址的 `KV` 值调用指针接收者方法（编译器自动取地址），但**符号上只有一个** `(*KV).Set`。省略星号时 Delve 会尝试补全，这是便利而非等价。
- **函数内行偏移与行表**：`:N` 由 Delve 在函数的行表条目上做偏移，所以它对源码改动不敏感；`文件:行号` 直接查行表，改代码就会漂。
- **`init` 的多实例问题**：Go 允许一个包里有多个 `init` 函数（甚至同一文件多个），它们都被编译成 `pkg.init.0`、`pkg.init.1`，函数名不足以区分位置，所以 `help break` 要求用 `文件:行号`。

6.8. 本次操作与当前状态

- 本机实测八组：`break db.(*KV).Set`（kv.go:48）、`break ...:2`（kv.go:50）、`break db.KV.Set`（也能解析）、`break Set`（ambiguous 并列出候选）、`break /Set/`（288 个断点）、`break /db\.\(\*KV\)\.Set/`（1 个）、`break setbp ...` + `on setbp ...`（命名与事后条件）、Neovim headless 里 `vim.fn.search` + `dap.set_breakpoint`（line 48 带条件）。
- 读取 `kv.go` 当前内容确认行号：`Set` 声明在 48 行，`make` 在 49 行，`bytes.Equal` 在 50 行（本人已把 `Get` / `Set` / `Del` 的 error 分支删掉）。
- 未改动 `internal/db` 任何文件；init 脚本与 nvim 探针在会话临时目录。
- 待本人动手：在 `dlv test ./internal/db` 里依次试 `break db.(*KV).Set`、`break db.(*KV).Set:2`、`break setbp db.(*KV).Set` + `on setbp string(key) == "k1"`，体会三种粒度的差别。

Index of Listings

Listing 1 未命中用 ok 表达、不产生 error（sync/map.go） 5
Listing 2 kv.go 第 51 至 55 行的现状 6
Listing 3 Set 的四步顺序：读、算、写、返回 6
Listing 4 零值未初始化时写 map 的实测结果 7
Listing 5 不拷贝也可以，但必须把「别名」与「有效期」写进文档 8
Listing 6 kv_test.go 第 39 行：三个条件焊成一条断言 8
Listing 7 每个 case 三个字段，断言分开写，失败消息自带 got 与 want 8
Listing 8 把三合一断言拆成三条，失败点一眼可见 9
Listing 9 三个红点，同一句失败信息 13
Listing 10 在第一条失败断言上停下，读三个值 14
Listing 11 第 36 行停下时的 updated、err 与 mem 14
Listing 12 在 Set 内部：先看参数，再走两步看返回值 15
Listing 13 args 给了参数，locals 却是空的 15
Listing 14 只在键是 k1 时停下，然后走两步看 make 的效果 16
Listing 15 同一变量的三次快照：垃圾值、nil、零字节 16
Listing 16 在 Del 内部逐行走，观察 mem 与 deleted 的先后 16
Listing 17 next 两次之后：删除已经发生，deleted 仍是 false 17
Listing 18 bt 的四层：测试函数、tRunner、子测试包装、goroutine 出口 17
Listing 19 本节所有输出都由这一条命令产生 18
Listing 20 adapter 就是一条 `dlv dap -l 127.0.0.1:PORT` 命令 22
Listing 21 等价于 `dlv test ./internal/db -- -test.run '^TestKVBasic$'` 22
Listing 22 Neovim 侧的 evaluate 与 CLI 的 `print updated` 得到同一个值 23
Listing 23 它用 treesitter 找光标最近的测试函数名，再拼 `-test.run ^名字$` 23
Listing 24 条件断点：与 CLI 的 `if string(key) == "k1"` 完全一致，不需要转义引号 23
Listing 25 放在 `~/.config/nvim/lua/plugins/` 下的 override； `build_flags` 也可以在这里加 24
Listing 26 `dlv help test` 的完整说明（节选） 29
Listing 27 测试二进制认识的 flag 由 testing 包自己注册（第 460、465 行） 30
Listing 28 正则语义与按 `/` 切分的子测试匹配规则 30
Listing 29 正则选中了 TestKVBasic； `-test.v` 让 RUN/FAIL 行出现 30
Listing 30 错误写法：测试参数没有分界 30
Listing 31 参数被当成了额外的包，编译阶段就失败 30
Listing 32 带不带前缀，取决于「谁在启动测试二进制」 31
Listing 33 测试二进制的 flag 包里没有 `-v` 这个名字 31
Listing 34 三个信号：0x7f 开头的地址、???、? 加 −1 的源码位置 34
Listing 35 `file` 的输出：动态链接、带调试信息、未 strip 34
Listing 36 开启了 cgo 的构建会链上系统 C 库，因此需要动态链接器 35
Listing 37 断点命中后，PC 落在 Go 代码里， `list` 立刻有内容 35
Listing 38 停在 `runtime.futex` 时， `list` 显示的是 `sys_linux_amd64.s` 35
Listing 39 `help break` 的位置描述符清单 39
Listing 40 Delve 把候选列出来，让你自己收窄——这就是「无歧义」的含义 40

Listing 41 不带偏移停在函数声明行；`:2` 停在函数体内第二行 40
Listing 42 命名断点与两种加条件的方式 41
Listing 43 给断点起名 `setbp`，之后用 `on` 补条件 41
Listing 44 正则里转义 `.` `(` `)`，只命中本项目的方法 41
Listing 45 `dset 'db.(*KV).Set' -test.run '^TestKVBasic$'` 即停在 Set 入口 41
Listing 46 函数名是第一个参数，其余参数原样交给测试二进制 42
Listing 47 本机 headless 实测：LINE=48，断点落在 48 行且带条件 42
Listing 48 `dap.breakpoints.get()` 的实际内容 42